IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA (DEEMED TO BE UNIVERSITY)

The Yenepoya institute of arts , science, commerce and management

Project – High Level Design

on

Natural Language Interface for E-commerce infrastructure

Student Name: Abhishek K K

Industry Mentor:Mr.shashank **Guided by: Ms.Yuktha**

Table of Contents

1. Introduction 3
1.1 Scope of the Document 3
1.2 Intended Audience 3
1.3 System Overview 3
2. System Design 4
2.1 Application Design 4
2.2 Process Flow 5
2.3 Information Flow 7
3. Components Design 7
4. Key Design Considerations 7
5. API Catalogue 8
6. Data Design 8
6.1 Data Model 8
6.2 Data Access Mechanism 9
6.3 Data Retention Policies 9
6.4 Data Migration 9
7. Interfaces 9
8. State and Session Management 11
9. Caching 11
10. Non-Functional Requirements 11
10.1 Security Aspects 11
10.2 Performance Aspects 11
11. References 12

1. Introduction

This document presents the High-Level Design for Natural Language Interface for E-commerce infrastructure— a comprehensive e-commerce application featuring both a command-line interface (CLI) and a browser-based web frontend, supported by a Flask REST API backend and JSON file persistence. The system enables users to browse products, manage shopping carts, and complete checkout transactions through either a terminal menu-driven interface or an interactive web SPA (Single-Page Application). The web frontend introduces user authentication, wishlist management, order history, and a multi-step checkout experience.

1.1 Scope of the Document

Covers architectural design of a three-tier application comprising two parallel frontend interfaces (CLI and Web SPA), a Flask REST API backend, and JSON file-based data persistence. Includes system components, API specifications, data models, interface descriptions, state management approach for both frontends, and non-functional requirements for the e-commerce platform.

1.2 Intended Audience

Software architects, backend developers, frontend developers (CLI and web), UI/UX designers, quality assurance teams, system administrators, and technical stakeholders involved in implementation, deployment, testing, and maintenance of the e-commerce application and its dual-interface architecture.

1.3 System Overview

The application follows a client-server architecture. Two parallel frontend clients communicate with the same Flask backend via HTTP REST API: (1) the CLI frontend (main.py) runs in a terminal and accepts numbered menu selections and natural language commands; (2) the Web frontend (templates/index.html + static/js/app.js) runs in a browser as a Single-Page Application served directly by Flask. Both clients call the same four REST endpoints. The web frontend additionally manages client-side state using localStorage for authentication, cart persistence, wishlist, and order history — functionality not present in the CLI. Backend manages product catalog, cart state, and order processing with JSON file persistence..

```
==================================
        ABHISHEK SMART STORE
==================================
1. Browse Products
2. View Cart
3. Checkout
4. Profile
5. Help
6. Exit

Select action [1-6]:
> Type number or command (e.g. 'show products')
> 6
Exiting Abhishek Smart Store.
(ecommerce) PS C:\Users\abhis\Desktop\Abhi-ecommerce\Abhi-ecommerce>
```

2. System Design

2.1 Application Design

Three-tier architecture with the presentation layer implemented as two parallel interfaces — a Python CLI module and an HTML/CSS/JavaScript SPA — business logic in a Flask REST API, and a data layer using JSON file storage. Both frontends send HTTP requests to the same backend endpoints. The backend processes requests, mutates the data store, and returns JSON responses. The web SPA additionally manages a client-side state layer in localStorage for features like authentication and order history. Separation of concerns enables independent development and testing of each component.

2.2 Process Flow

CLI Flow: User launches main.py → branded menu displayed → user selects action via number or natural language → CLI parses input and routes to flow handler → handler sends HTTP request to Flask endpoint → backend validates, processes, updates JSON → response returned → CLI formats and displays result → returns to main menu loop.

Web Frontend Flow: User opens browser at http://127.0.0.1:5000 → Flask serves index.html → JavaScript initializes app state from localStorage → unauthenticated users routed to Login page → after login, user routed to Store page → products fetched from GET /products → user adds items to cart (POST /add called per item) → user proceeds to Checkout page → delivery and payment details entered → Place Order calls GET /checkout on backend → order saved to localStorage history → success screen shown.

```
(ecommerce) PS C:\Users\abhis\Desktop\Abhi-ecommerce\Abhi-ecommerce> python main.py
========================================
1. Browse Products
2. View Cart
3. Checkout
4. Profile
5. Help
6. Exit

Select action [1-6]:
> Type number or command (e.g. 'show products')
> 1

----------------------------------------
PRODUCT LIST
----------------------------------------
ID  Name           Price     Stock
1   Laptop         Rs.50000  10
2   Phone          Rs.20000  9
3   Headphones     Rs.2000   13
----------------------------------------
Select product (ID/name) or 'back': 2

----------------------------------------
PRODUCT DETAILS
----------------------------------------
Name     : Phone
Price    : Rs.20000
Stock    : 9
Delivery : 2 days
Specs    : 6GB RAM
----------------------------------------
1. Add to Cart
2. Back to Product List
Select action [1-2]:
```

2.3 Information Flow

Product catalog flows from data.json → Flask backend → both CLI and Web frontend for display. In the CLI, product data is formatted as a terminal table. In the web frontend, products are rendered as styled cards in a responsive grid. User cart actions (add item, quantity change) flow from either frontend as POST /add requests. Cart state is maintained in data.json by the backend. The web frontend maintains a parallel local copy in localStorage for immediate UI updates. Checkout operation (GET /checkout) clears the backend cart. The web frontend additionally saves order details — order ID, date, items, total — to localStorage for order history. Stock levels are decremented atomically on successful add operations in both flows.

3. Components Design

CLI Frontend Component (main.py): Menu rendering, input parsing with NLP support, HTTP client for backend communication using requests library, response formatting, flow orchestration across product browsing, cart management, checkout, and profile views.

Web Frontend Component (index.html + app.js + style.css): Single-page application with client-side router, six page views (Login, Signup, Forgot Password, Store, Checkout, Profile), state management via global state object and localStorage, API integration layer via Fetch API, toast notification system, cart drawer, product grid with search/filter/sort, and multi-step auth flows.

Flask Backend Component (app.py): REST API server, request validation, business logic execution, JSON file I/O operations, static file serving and Jinja2 template rendering for the web frontend.

Data Storage Component: JSON file with products and cart arrays, atomic read-write via load() and save() helper functions.

HTTP Communication Layer: requests library for CLI, Fetch API for web frontend, both targeting localhost:5000 with JSON payloads.

4. Key Design Considerations

Separation of Concerns: CLI handles terminal presentation, web frontend handles browser presentation, Flask backend manages business logic, data layer ensures persistence. Both frontends are independently usable.

Dual Interface Architecture: The same Flask backend serves both a CLI client and a web SPA. This demonstrates a clean API-first design where the business logic is fully decoupled from the presentation layer.

Hybrid Input Support (CLI): Menu numbers and natural language commands provide flexible user experience in terminal mode.

Client-Side State (Web): Authentication, wishlist, and order history are managed client-side in localStorage as a prototype-appropriate approach, without requiring additional backend endpoints.

Stock Validation: Atomic stock checks in the backend prevent overselling regardless of which frontend initiates the request.

Graceful Error Handling: Network failures and invalid inputs handled with user-friendly messages — print statements in CLI, toast notifications in web frontend.

Stateless API: Backend endpoints are stateless with file-based state management, making them consumable by any HTTP client.

Auth Scope: CLI operates without authentication (single-user, localhost). Web frontend adds client-side authentication with localStorage user store as a demonstration layer.

5. API Catalogue

All four endpoints are consumed by both the CLI frontend and the Web frontend.

GET /products — Returns array of product objects. Each object contains: name (string), price (integer, INR), stock (integer), delivery (string), specs (string). Used by CLI product_flow() and web loadProducts().

GET /cart — Returns array of cart items. Each item contains: product (string, product name reference), qty (integer). Used by CLI cart_flow() and web renderCartDrawer().

POST /add — Accepts JSON body: { "product": string, "qty": integer }. Validates product exists, qty is positive integer, sufficient stock available. Decrements stock, appends to cart, persists. Returns: { "message": string } on success or error. Used by CLI product_flow() and web addToCart().

GET /checkout — Validates cart is not empty, sets cart to empty array, persists. Returns order confirmation message. Used by CLI checkout_flow() and web placeOrder().

6. Data Design

6.1 Data Model

Product Model: name (string, unique identifier), price (integer, INR), stock (integer, mutable available quantity), delivery (string, estimated delivery time), specs (string, technical specifications). Cart Item Model: product (string, references product name), qty (integer, ordered quantity). JSON Root Structure: { "products": [...product objects], "cart": [...cart item objects] }. Web Order Model (localStorage only, not in data.json): id (string, format #SSxxxx), date (string, localeDateString), items (array of cart items), total (integer), status (string: 'processing'/'delivered').

6.2 Data Access Mechanism

Backend implements load function reading entire JSON file into memory and save function writing complete data structure back to disk. File path resolved relative to backend script location ensuring portability. Read operations occur at each API request, write operations occur after state mutations. No database or ORM layer required for current single-user scope.

6.3 Data Retention Policies

Product catalog persisted in data.json indefinitely with manual updates. Cart state retained in data.json until explicit checkout or manual reset. Web frontend cart mirrored in localStorage under key ss_cart and cleared on checkout. Web frontend order history persisted in localStorage under ss_orders without expiry — retained until browser data is cleared. User accounts stored in localStorage under ss_users. No server-side order history or user account persistence currently implemented.

6.4 Data Migration

Not applicable for current JSON file-based approach. Future migration to relational database would require schema definition, data transformation scripts, and backward compatibility handling for existing JSON data files.

7. Interfaces

Command Line Interface: User-facing terminal menu system with hybrid NLP input parsing, table-formatted product displays, cart summaries with totals, confirmation dialogs. Runs as a Python script in a separate terminal from the backend.

Web SPA Interface: Browser-based single-page application with six pages: Login (email/password form with demo login), Signup (two-step with OTP), Forgot Password (three-step reset), Store (hero section, search bar, category filters, product grid), Checkout (delivery form + payment method + order summary), Profile (order history, settings, wishlist, addresses). Served by Flask at root URL.

REST API Interface: Four HTTP endpoints on localhost port 5000 accepting GET and POST requests with JSON payloads. Consumed by both CLI (via requests library) and Web frontend (via Fetch API).

File System Interface: JSON file read-write operations for product catalog and cart state. Handled by Flask backend load() and save() functions.

localStorage Interface (Web only): Browser key-value storage for user accounts (ss_users), current session (ss_user), cart (ss_cart), wishlist (ss_wishlist), orders (ss_orders). Managed by saveState() function in app.js.

```
Choose next step:
1. Return to Main Menu
2. Exit
Select action [1-2]: 1

========================================
         ABHISHEK SMART STORE
========================================
1. Browse Products
2. View Cart
3. Checkout
4. Profile
5. Help
6. Exit

Select action [1-6]:
> Type number or command (e.g. 'show products')
> 2

----------------------------------------
YOUR CART
----------------------------------------
Item            Qty     Subtotal
Phone           x1      Rs.20000
----------------------------------------
TOTAL: Rs.20000

Choose next step:
1. Return to Main Menu
2. Exit
Select action [1-2]:
```

```
========================================
         ABHISHEK SMART STORE
========================================
1. Browse Products
2. View Cart
3. Checkout
4. Profile
5. Help
6. Exit

Select action [1-6]:
> Type number or command (e.g. 'show products')
> 5

----------------------------------------
HELP
----------------------------------------
Actions:
- show products / list / view items / 1
- cart / 2
- checkout / 3
- profile / 4
- help / 5
- exit / 6
----------------------------------------

Choose next step:
1. Return to Main Menu
2. Exit
Select action [1-2]:
```

8. State and Session Management

CLI: Application state maintained in data.json as single source of truth. Backend API operates statelessly. CLI maintains ephemeral state only for current menu navigation. No user session.

Web Frontend: Two-layer state management. Server-side state: product catalog and cart in data.json via Flask API. Client-side state: global JavaScript state object (state.user, state.cart, state.wishlist, state.products, state.searchQuery, state.sortBy, state.categoryFilter) synchronized to localStorage on every mutation via saveState(). Session is established on login by storing user object to localStorage and terminated on logout by removing it. Cart badge, navbar auth buttons, and protected pages (Checkout, Profile) all derive their state from the client-side state object.

9. Caching

No explicit server-side caching layer implemented. Each API request reads fresh data from data.json. The web frontend implicitly caches the product catalog in state.products after the initial GET /products call — subsequent renders (search, filter, sort) operate on the in-memory array without re-fetching, until the store page is navigated to again. Cart state in the web frontend is maintained in both the client-side state object and localStorage, providing an immediate-response user experience for cart badge updates and drawer rendering without waiting for API calls. Python module import caching leveraged automatically by the interpreter.

10. Non-Functional Requirements

10.1 Security Aspects

Input Validation: Product names and quantities validated on backend before processing. Stock checks prevent inventory overselling. Localhost Binding: Flask server binds to 127.0.0.1 limiting network exposure. Web Auth Limitation: Client-side authentication using localStorage stores passwords in plain text — this is a prototype scope limitation and must be replaced with server-side session tokens or JWTs in any production deployment. OTP flows in Signup and Forgot Password are client-side simulations suitable for UI demonstration only. No HTTPS configured for localhost development. File Permissions: data.json readable and writable by application process.

10.2 Performance Aspects

Response Time: API endpoints respond within milliseconds for product catalogs under 100 items. Web frontend initial load time is negligible as all assets are served locally. File I/O: JSON read-write operations complete in single-digit milliseconds. Network Latency: Localhost communication eliminates network latency for both CLI and web frontend. Web Rendering: Product grid renders client-side with animation staggering (60ms delay per card). Search and filter operations run synchronously on the in-memory products array, providing instant results without API round-trips. Scalability: Current architecture suitable for demonstration and single-user scenarios.

11. References

Mozilla Developer Network — Fetch API Documentation: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API.

Web Storage API Specification — localStorage/sessionStorage. Flask Template Rendering (Jinja2) Documentation. Single-Page Application Architecture Patterns and Best Practices.

CSS Custom Properties (Variables) — W3C Specification.